Advanced Computer Architecture

Chapter #1

Introduction to Parallel Processing

1. Introduction to Parallel Processing

Parallel Computer Structures

Architectural Classification Parallel Processing Applications

Need of parallel programming, characteristics of parallel algorithms

Parallel programming techniques

Parallel programming models

Parallel algorithms for multiprocessors

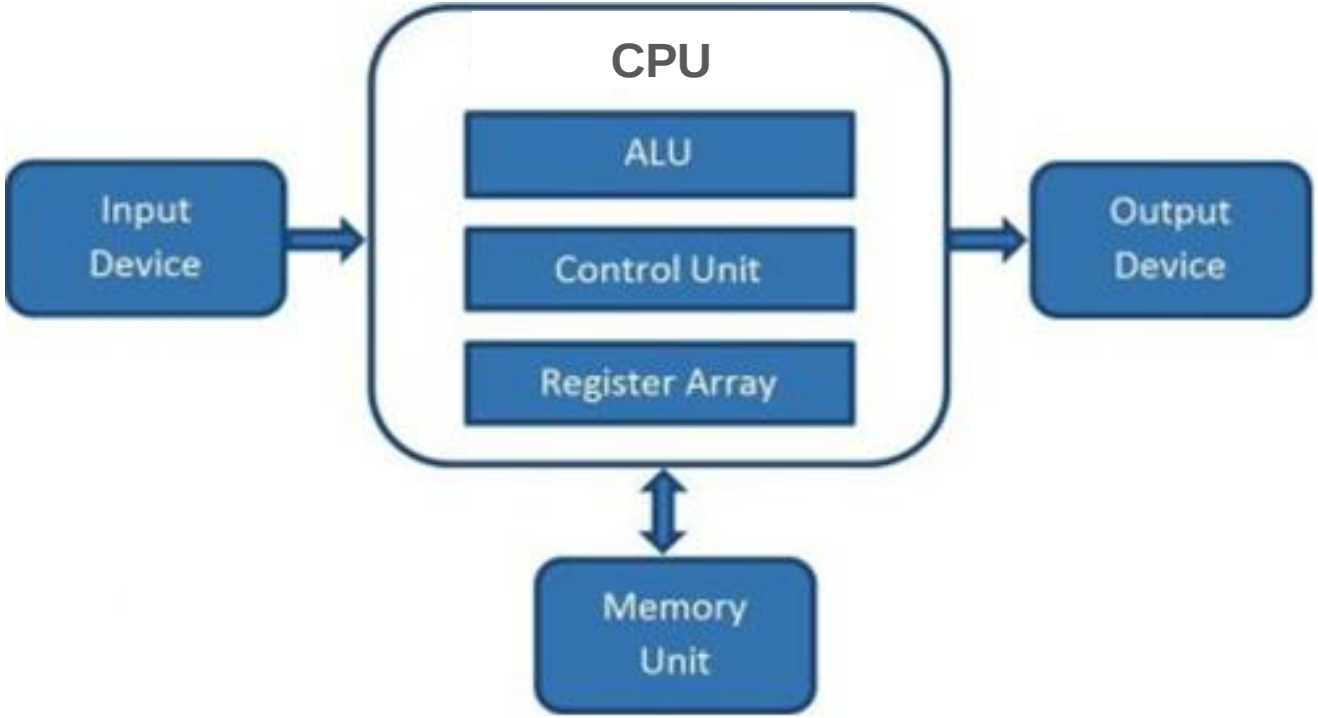
Performance of parallel algorithms

Parallel programming languages

Problems based on parallel algorithms

Basic Structure of a Computer

Basic Structure of a Computer



Conventional Architecture

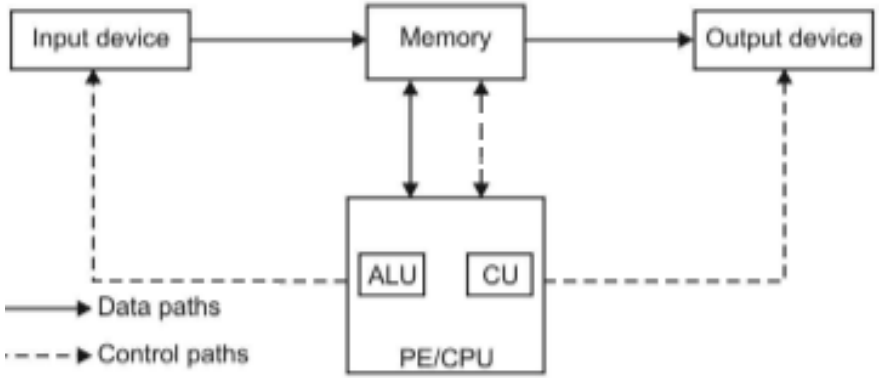
Von Neumann's Architecture

A computer consists of a CPU, memory and input/output unit

CPU is the main processing element ⇒ microprocessor

CPU is composed of ALU, CU and Registers

Von Neumann's Architecture



A computer consists of an input unit, an output unit, a program memory and a CPU or microprocessor also named as PE

In 1940 John Von Neumann proposed a stored-program concept based computer that is also known as a sequential computer or a uniprocessor or Von-Neumann's machine or the conventional computer

The PE received on instruction of the program at a time ⇒ interprets is and execute it

Its operation is sequential

Von Neumann’s Architecture

The PE can execute only one instruction at a time

The speed is limited by the speed at which PE can retrieve instructions and data from memory and the speed at which it can process retrieved data

Data & Instruction from the same memory ⇒ Access conflict ⇒ Bottleneck

To increase the speed of processing of data

- Increase the speed of the PE **OR**
- Interconnect many such sequential computers to work together

⇒ Parallelism can address the issue

Machine based Computation

Few year back:

- Computer cannot have any intelligence
- It can not do anything “by itself”
- It can only follow the code it has - step by step
- The program tells it what to do - hence it is doing what it is told to do

Augmenting Human Capabilities: Computers can augment human capabilities by providing tools for **data visualization**, **communication**, and **problem-solving**

AI: Realizing several AI applications

High processing power of computer

Processing Power of Computer

High CPU performance ⇒ always desirable ⇒ but not easy

Approaches for improving the computer performance:

- Enhancing the performance of a single computer
- Use of parallelism / parallel processing

Parallelism can be done:

- Within a **CPU** ⇒ Uniprocessor system
- With **multiple CPUs** / processing devices ⇒ Multi-processor system

Parallel Processing

Parallelism

When more than one tasks are performed simultaneously ⇒ **parallelism**

Computer which consists of a number of interconnected sequential computers which cooperatively execute a single program to solve a large problem ⇒ **parallel computer**

A system that processes two or more different instructions simultaneously ⇒ **parallel processing**

Conventional Architecture

Features and Constraints

Uniprocessor System

Initially a computer has a single processing unit (CPU) ⇒ microprocessor

The single CPU is responsible to execute each of the task

Only one CPU ⇒ one program in execution at any instant ⇒ can execute only one instruction at a time

Modern software made to use multiprocessing architectures ⇒ now most desktop computers are with multiprocessing architectures ⇒ time sharing

Architecture is based on a single computing unit

In this system all operations/manipulations (additions, multiplications, comparison etc.) are done sequentially on the unit

Parallelism

The parallelism is execution of multiple events concurrently in the computing process to achieve faster computational speed or better throughput

Traditionally, the computer has been viewed as a sequential machine

Most computer programming languages require the programmer to specify algorithms as a sequence of instructions

Each instruction is executed in a sequence of operations like:

Fetch instruction ⇒ Fetch operands ⇒ Perform operation ⇒ Store results

Processors execute programs by executing machine instructions in a sequence one by one ⇒ but the view of the computer as complete sequential system has never been entirely true, such as:

- At micro-operation level, multiple control signals are generated simultaneously
- Instruction pipelining, uses overlapping fetch and execute operations
- Superscalar organization, which exploits instruction-level parallelism

Enhancements in Uniprocessor System

Performance Enhancement

Different ways for improving the performance:

- Architectural Aspects:
  - Pipelining
  - Superscalar ⇒ multiple execution units
  - Cache memory ⇒ Memory hierarchy
- Clock Speed: Using higher clock speed allows to execute instructions faster
- Fabrication Process Technology: Using smaller transistors ⇒ dense & fast
- Software Optimization
  - Compilers: Efficient compilers can generate optimized code as per processor's capabilities
  - Programming Techniques: Careful programming practices can reduce the number of instructions executed and improve memory access patterns

Parallelism in Uniprocessor System

To achieve parallelism in uniprocessor system a multiple hardware functional units operating in parallel inside a single processor

The instruction dispatch unit assigns the current instruction to the relevant unit based on the decoding result

The assigned unit continued with the steps such as

- Operand address calculation
- Operand fetch
- Execution etc.

Parallelism in Uniprocessor System

When the current instruction is assigned to the functional unit the instruction dispatcher unit assigns next instruction to an another unit when it is free

It can have more than one functional unit of the same type like multiple adders, multiple shifter etc.

The control unit is more complex than that of the simple processor

However the throughput achieved is much higher

The control unit should be alert in conflict handling such as dependency cases

Performance Enhancement in Uniprocessor System

Multiprogramming & time-sharing

Multiple functional units

Harvard Architecture

Memory Hierarchy

Pipelining

Multiprogramming & Time-sharing

Multiprogramming ⇒ several processes reside in main memory and CPU switches from one process (say P1) to another (say P2) when the currently running process (P1) is blocked for some I/O operation

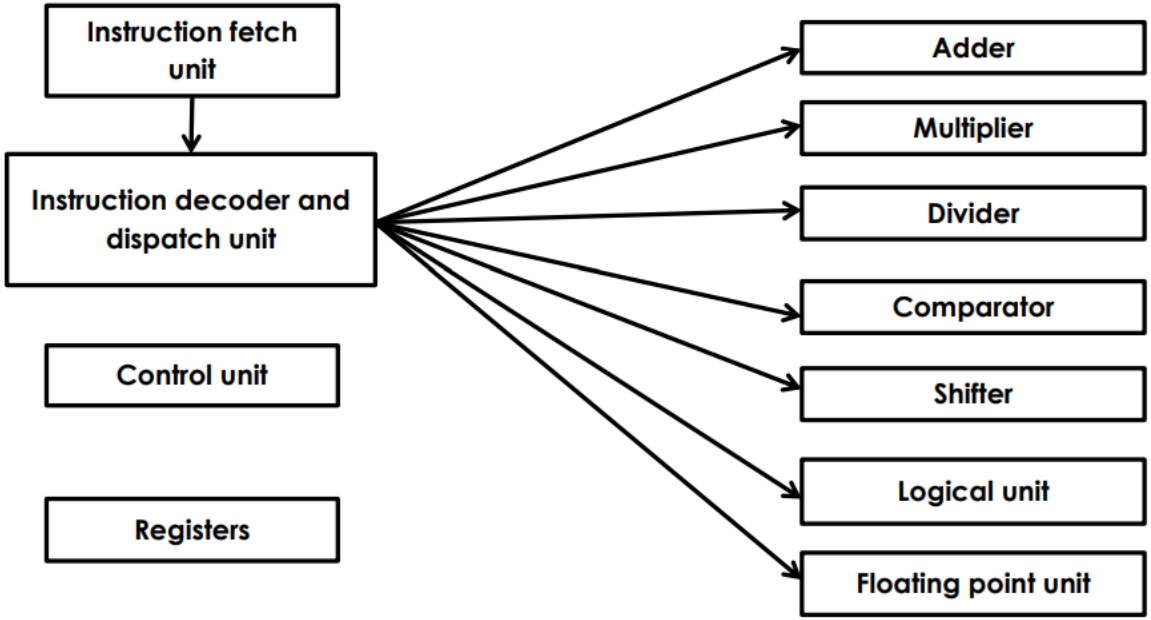
I/O operation for P1 is handled by a DMA unit while the CPU is running P2

In time-sharing processes are assigned slices of CPU's time

The CPU executes the processes one by one on the basis of predefined algorithm like round robin ⇒ but in long run it seems to be all processes are running simultaneously ⇒ overall increased throughput

Multiple Functional Units

Use of multiple functional units like multiple adders, multipliers or even multiple ALUs to provide concurrency is not a new idea in uniprocessor environment; it has been around for decades

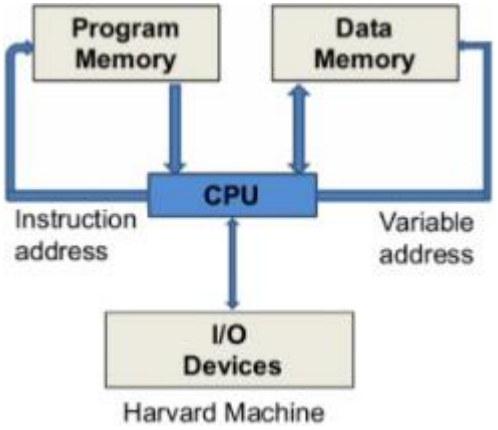


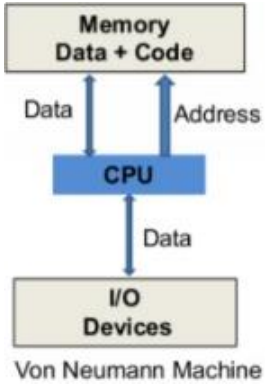
Harvard Architecture

Provides separate memory units for instructions and data

Effectively doubles the memory bandwidth saving CPU's time, ⇒ in split cache instructions are kept in I-cache and data in D-cache

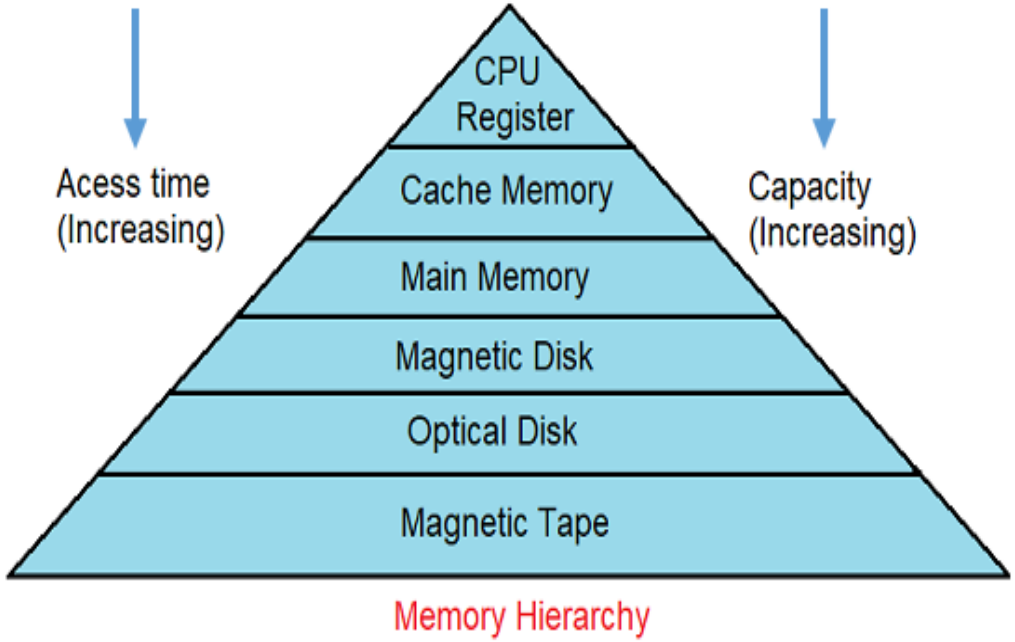
In contrast when instructions and data are kept in the same memory, the architecture is called Princeton Architecture ⇒ unified cache, main memory





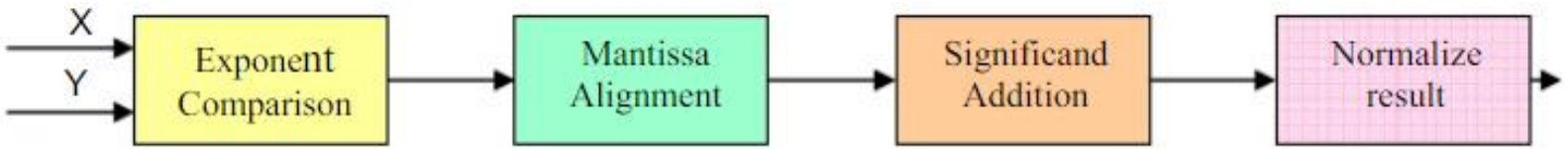
Memory Hierarchy

A parallel processing mechanism supported by memory hierarchy is the simultaneous transfer of instructions/data between (CPU, cache) and (Main Memory, Secondary Memory)



Pipelining

Arithmetic Pipeline:



Four stage floating point Adder

Instruction Pipeline:



Five stage instruction pipeline

Uniprocessor System

Techniques for performance enhancement of uniprocessor system:

- Increase parallelism within uniprocessor system: Harvard architecture, Pipelining, Multitasking
- Cache memory & Memory hierarchy
- Increase the operating clock frequency
- Decrease the size of components

Limitations of Uniprocessor System

Limitations

Only single instruction execution at a time

Relatively slower

Difficult and quite expensive to speedup

The speedup process is reached close to saturation level

Need of Parallel Computing

Speed is proportional to the speed of data that can move through hardware

Speed is limited by the speed at which PE can retrieve instruction and data from the memory

Whatever level of parallelism and advancements within uniprocessor system the Instructions execution rate is limited

Parallel Computing

A type of computing architecture in which several processors simultaneously execute multiple, smaller calculations broken down from an overall larger, complex problem

Naturally, a multiprocessor system ⇒ that can execute more than one instructions simultaneously

Sequential	vs.	Parallel Computer
Uniprocessor systems ⇒ 1 CPU or PE (Processing Element)		Multiprocessor systems ⇒ many CPUs or PEs
Can execute only one instruction at a time		Can execute multiple instructions at a time
Speed is limited		There is no limitation of speed
Difficult and expensive to make faster		More practical approach to achieve better performance

Multiprocessor System

Multiprocessor system

More practical approach

Many CPUs/processing units

Multiple programs are in execution simultaneously

Can execute several instructions at a time

No limitation on instruction execution rate

Less expensive and feasible to make it faster ⇒ But there are additional challenges to be addressed

Parallel Processing

Multiprocessor System

A multiprocessor system is an interconnection of two or more CPUs with memory and input-output equipment

In multi computer system computers are interconnected with each other to form a network and they may or may not communicate with each other

A multiprocessor system is controlled by one operating system that provides interaction between processors and all the components of the system cooperate in the solution of a problem

Multiprocessing improves the reliability of the system so that a failure in one part has a limited effect on the rest of the system

Multiprocessor System

The computations can proceed in parallel as:

- Multiple independent jobs can be made to operate in parallel
- A single job can be partitioned into multiple parallel tasks

Multiprocessing can improve performance by decomposing a program into parallel executable tasks in in different ways

- The user can explicitly declare that certain tasks to be executed in parallel
- Use a compiler that can automatically detect parallelism in a user's program

⇒ Compiler checks for data dependency in the program, and if a program depends on data generated in another part, the part yielding the needed data must be executed first

Parallel Computing and Its Features

Parallel Computing

Many computations or processes are carried out simultaneously

Involves breaking down a large problem into smaller, independent parts that can be solved simultaneously by multiple processors

Parallelism is ability to execute multiple tasks simultaneously

Concurrency gives the ability to handle multiple tasks at the same time, even if they are not executing simultaneously

Processing elements are the individual units within a computer system that can perform computations, such as **CPUs** or **GPUs**

Features of Parallel Computing

Increased performance: By distributing the workload across multiple processors, parallel computing can significantly speed up the execution of large and complex tasks

Scalability: Parallel computing systems can be easily scaled by adding more processing elements to handle larger workloads

Reduced execution time: Parallel computing can significantly reduce the time it takes to complete a task, especially for computationally intensive problems

Needs
of
High Processing Power
or
High Performance Computing

Needs of High Performance Computing

Weather forecasting

Environmental modeling

Astrophysics

Aerodynamics

AI & ML

Computer Vision

Scientific Research

Simulating complex weather patterns and climate change to analyze large datasets and predict future scenarios

Designing a new drug involves simulating molecular interactions and predicting their effectiveness

Developing new materials with specific properties often involves complex simulations and calculations

To model the interaction between atoms and understand micro-behaviour of materials quantum and structural chemists

Artificial Intelligence and Machine Learning

Deep Learning - Training deep neural networks, which are at the core of many AI

Natural Language Processing - Understanding and generating human language involves complex algorithms and large datasets

Computer Vision - Recognizing and interpreting images and videos requires sophisticated algorithms and high-performance hardware to process and analyze the data

Other Applications

Financial Modeling - executing trades in milliseconds, assessing & mitigating financial risks

Video Editing and Rendering - Creating high-quality videos and visual effects

3D Modeling and Animation - Designing and rendering complex 3D models and animations

Video Games - Developing and playing modern video games with realistic graphics and physics simulations

Virtual Reality and Augmented Reality: Creating immersive virtual and augmented reality experiences

Other Applications

Data Analytics and Visualization: Analyzing and visualizing large datasets to extract insights and make informed decisions

For realistic graphic images for simulation of aircraft takeoff, landing etc.

To save time and cost - As parallel computing is very fast and multiple cheap computing resources can reduce cost as compared to expensive single one

To provide concurrency i.e., parallel execution of tasks

Classification of Parallel Computers

Flynn’s classification

Structural classification

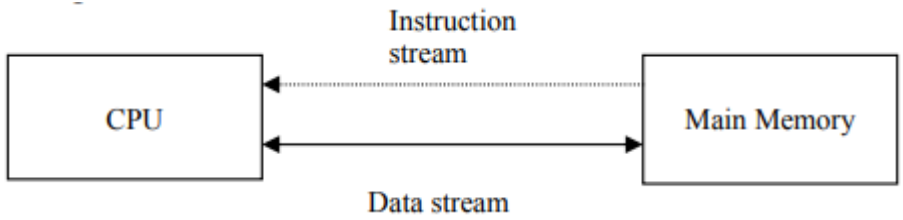
Multiple Processor Organizations

Types of Parallel Processor Systems

A taxonomy first introduced by Flynn is still the most common way of categorizing systems with parallel processing capability

Flynn proposed different categories of computer systems

Flynn’s Classification



Architectural Classification: Flynn's classification

Based on the multiplicity of Instruction Streams and Data Streams

- Instruction Stream: Sequence of Instructions read from memory
- Data Stream: Operations performed on the data in the processor

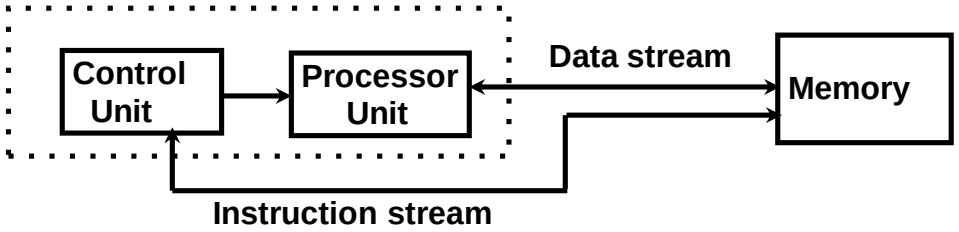
		Number of *Data Streams*	
		Single	**Multiple**
Number of *Instruction Streams*	**Single**	**SISD**	**SIMD**
	Multiple	**MISD**	**MIMD**

SISD

Single instruction, single data (SISD) stream

A single processor executes a single instruction stream to operate on data stored in a single memory

Uniprocessors fall into this category



SISD Characteristics & Limitations

Characteristics

- Standard Von Neumann machine
- Instructions and data are stored in memory
- One operation at a time

Limitations

- Von Neumann bottleneck
- Maximum speed of the system is limited by the Memory Bandwidth (bits/sec or bytes/sec)
- Limitation on Memory Bandwidth Memory is shared by CPU and I/O

SISD Performance Improvements

Multiprogramming

Multifunction processor

Pipelining

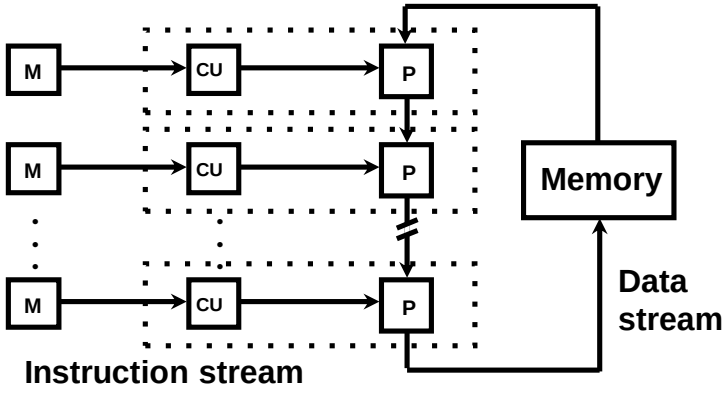
Instruction-level parallelism ⇒ Superscalar

MISD (Multiple Instruction, Single data) Stream

A sequence of data is transmitted to a set of processors, each of which executes a different instruction sequence

Not commercially implemented structure

There is no computer at present that can be classified as MISD



SIMD (Single Instruction, Multiple Data) stream

A single machine instruction controls the simultaneous execution of a number of processing elements on a lockstep basis

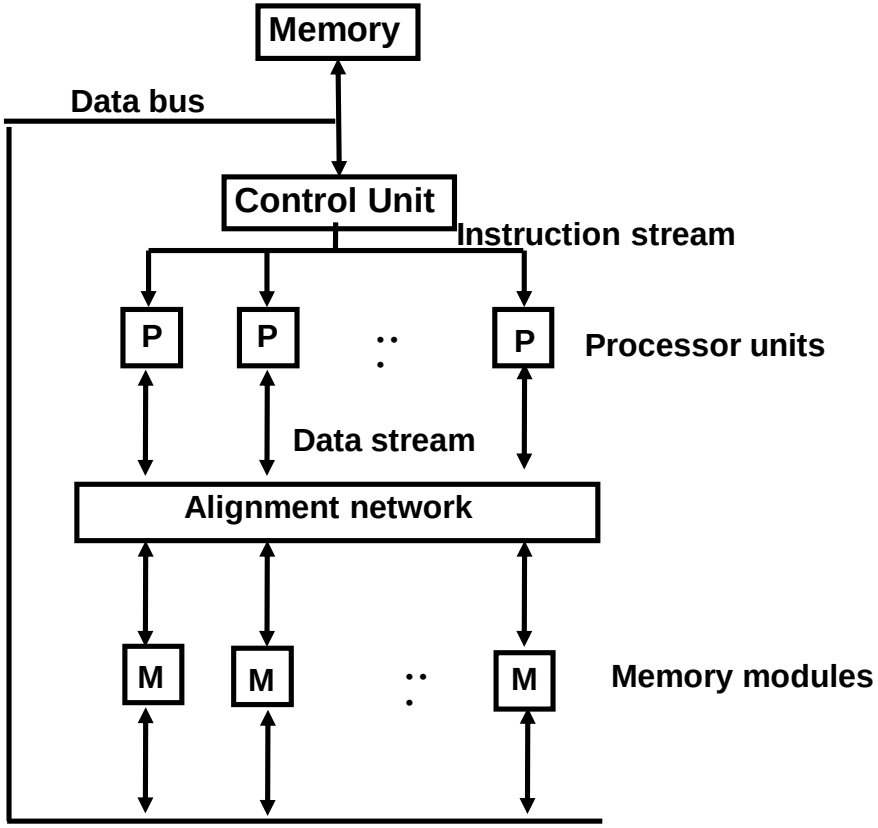
Each processing element has an associated data memory, so that each instruction is executed on a different set of data by the different processors

Vector and array processors fall into this category

Characteristics

- Only one copy of the program exists
- A single controller executes one instruction at a time

SIMD Structure



MIMD (Multiple Instruction, Multiple Data) stream

A set of processors simultaneously execute different instruction sequences on different data sets

SMPs (Symmetric Multiprocessors), clusters, and NUMA (Non-uniform Memory Access) systems fit into this category

With the MIMD organization, the processors are general purpose; each is able to process all of the instructions necessary to perform the appropriate data transformation

MIMDs can be further subdivided on the basis of processor communication

MIMD

If the processors share a common memory, then each processor accesses programs and data stored in the shared memory, and processors communicate with each other via that memory

The most common form of such system is known as a symmetric multiprocessor (SMP)

In an SMP, multiple processors share a single memory or pool of memory by means of a shared bus or other interconnection mechanism; a distinguishing feature is that the memory access time to any region of memory is approximately the same for each processor

MIMD

A more recent development is the non-uniform memory access (NUMA) organization,

As the name suggests, the memory access time to different regions of memory may differ for a NUMA processor

A collection of independent uniprocessors or SMPs may be interconnected to form a cluster

Communication among the computers is either via fixed paths or via some network facility

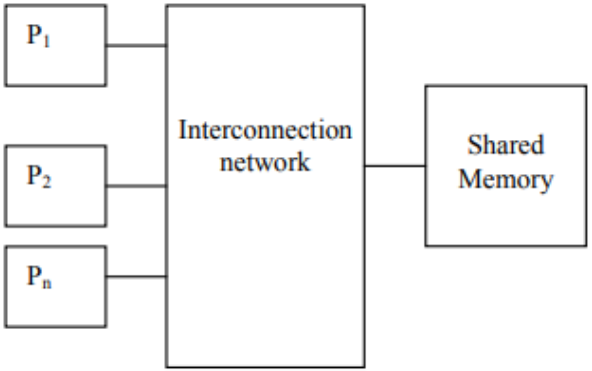
MIMD Computer Systems

Characteristics

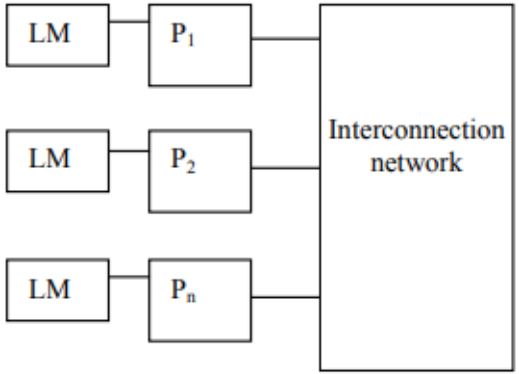
- Multiple processing units
- Execution of multiple instructions on multiple data

Types of MIMD computer systems

- Shared memory multiprocessors (tightly coupled)
- Message-passing multicomputers (loosely coupled)
- Hybrid



Tightly coupled system



Loosely coupled system

Parallel Computing Architectures

Shared Memory Systems

Distributed Memory Systems

Hybrid Systems

Shared Memory Systems

Multiple processors access the same physical memory ⇒ **tightly coupled system**

Information can be passed by placing that in common global shared memory

Doesn’t preclude each processor from having its own local memory

Most of the commercial tightly coupled multiprocessors have cache memory with each CPU

In addition there is a global common memory that can be accessed by all CPU

Allows for efficient communication between processors because they directly read from and write to a common memory space

Typically easier to program than distributed memory systems due to the simplicity of memory access ⇒ can face challenges with scalability and memory contention

Distributed Memory Systems

Another type of multiprocessor system is the distributed memory

Consist of multiple processors, each with its own private memory ⇒ **loosely-coupled system**

Each processor element has its own private local memory

Processors communicate by passing messages over a network

The processors relay program and data to other processors in packets

Can scale more effectively than shared memory systems, as each processor operates independently, and the network can handle communication between them ⇒ main challenge is the complexity of communication and synchronization

Hybrid Systems

Combine elements of shared and distributed memory architectures

They typically feature nodes that use shared memory, interconnected by a distributed memory network

Each node operates as a shared memory system, while communication between nodes follows a distributed memory model

Within a node, tasks can communicate quickly using shared memory, while inter-node communication uses message passing

Used in large-scale scientific computing, where computations are divided into smaller tasks within nodes and coordinated across a larger network

Can efficiently handle complex workloads that require both high-speed local computation and distributed processing across a large number of processors

Parallel Algorithms and Programming

Parallel Algorithms and Parallel Programs

Parallel Algorithm can execute a piece of program at a time on many different processing devices and combine together to get the correct result

Parallel Program has partitioning and distribution of main task or data ⇒ there is a workload distribution or data load distribution

Workload is distributed among multiple processors in the form of task ⇒ **Functional Parallelism**

Data structure is partitioned and distributed among multiple processors for parallel execution ⇒ **Data Parallelism**

Parallel Program ⇒ Functional parallelism or Data parallelism or Combination of both

Need of Parallel Programming

Need of Parallel Programming

Parallel computing ⇒ highly increases the throughput of the system

Many applications such as scientific computations, signal/image processing, data mining etc. ⇒ implemented with parallel computing

But parallel computing needs supporting hardware architecture ⇒ **parallel architecture**

Parallel architecture ⇒ allows concurrent execution of multiple processes or program units

Characteristics of Parallel Algorithms

Characteristics of Parallel Algorithms

Deterministic - Only deterministic algorithms are implementable

Computational granularity - size of data & program

Classified as fine grain / medium-grain / coarse-grain

Distribution of degree of parallelism in an algorithm

Communication patterns and synchronization requirements - Communication pattern ⇒ both memory and interprocessor communications between processors, and the patterns can be static or dynamic:

- Static algorithms ⇒ suitable for SIMD or pipelined machines
- Dynamic algorithms ⇒ suitable for MIMD

Synchronization frequency ⇒ affects the efficiency of an algorithm

Characteristics of Parallel Algorithms

Uniformity - Refers the types of fundamental operations to be performed

- If the operations are uniform across the data-set ⇒ SIMD or pipelining is more desirable due to better efficiency
- Randomly structured algorithms ⇒ MIMD are more suitable

Memory requirements and data structures - Used data structure and data movement pattern ⇒ affect efficiency of the program

Time and Space complexities ⇒ key measures of granularity of parallel algorithms

Parallel Programming Techniques

Basic approaches in parallelizing the program:

Explicit parallel programming

Implicit parallel programming

Explicit Parallel Programming

Defined as a parallelism technique where concurrent operations are executed parallel with the help of primitives that are known as special purpose directives or function calls

Programmer develops a parallel program by explicitly specifying the parallelism

Program need the parallel algorithm for given problem

The compiler does not detect the parallelism for the allocation of resources

The programming efforts required in explicit parallelism are more as compared to implicit parallelism

Explicit Parallel Programming

Different techniques that can be used by the programmer to explicitly make the parallel program:

- Use of specific programming languages that support parallel programming
- Use of compiler directives for library function calls
- Use of automatic parallelization tools that analyze the program and find parallelism

Examples:

- Parallel loops: A construct that allows a loop to be executed in parallel
- Data parallelism: A technique that is used to divide the present large datasets into smaller subsets ⇒ these divided subsets can then be processed in parallel

Implicit Parallel Programming

In implicit parallel programming a compiler or interpreter automatically exploit the parallelism inherent to the computations expressed by some of the programming language's constructs

No burden on programmer about the parallel programming languages and underlying parallel architecture

Programmer is relieved from parallelizing the program

Compiler is responsible for extracting the parallelism in the program

Compiler plays an important role for parallelizing

Examples: Pipelining, Multithreading etc.

Implicit vs Explicit Parallelism

parallel programming that automatically allow the compiler or interpreter to exploit the parallelism

The source program is coded sequentially and then translated into parallel object code by a parallelizing compiler

Parallelism is detected by the compiler and then assigns the resources to the target machine code

Requires fewer programming efforts by the programmers as compared to explicit parallelism

parallel programming that execute the concurrent computations with the help of primitives that are in the form of special purpose directives

Parallelism is explicitly specified in the source code itself

Parallelism is specified explicitly, there is no need for the compiler to detect parallelism, and resources are allocated explicitly

Requires more programming efforts by the programmers as compared to implicit parallelism

Implicit & Explicit Parallelism

Two ways of parallel programming, to achieve parallelism

Parallelism is carried out by the compiler or manually by coding

Parallel Programming Models

Parallel Programming Models

Used by the application programmer to develop parallel program

Programming model ⇒ specify

- How processes or parts of the parallel program communicate with each other
- What synchronization techniques are available to coordinate their activities

Provides a paradigm to the programmer for sharing, communication and synchronization among process of a parallel program

Parallel programming models for explicit parallel programming

- ❖ Message passing programming
- ❖ Shared memory programming
- ❖ Data parallel programming

Selection of Parallel Programming Models

Which model to select?

For parallel programming, we have to consider the underlying parallel architecture type and the model of interprocessor communication

Parallel architecture ⇒ interprocessor communication ⇒ using either **shared memory** or **distributed memory**

Development of parallel program depends on the basis of parallel architecture with shared or distributed memory ⇒ accordingly the programming model is selected

Message Passing Programming

Applicable for distributed memory parallel architecture in which each PE has local memory but no global shared memory

This model is used to develop parallel program for parallel architecture with distributed memory

Assumption - program consists of multiple processes with local variables

These processes communicate with each other by explicit coding

Programmer explicitly puts **'send**' and **'receive**' commands in the source code

Message passing programming may be **synchronous** or **asynchronous**

Synchronous Message Passing

Requires synchronization between sending message and receiving message

The command '**Blocked Send**' is explicitly used for sending message to another processor and the sending processor waits till the receiving processor has not received the message

After proper reception, the sending processor can continue its operations

It is used when the data or messages are to be send in a proper order and to ensure that it has reached to the processor

To send data, '**Blocked Send**' command is used in the source code of the sending processor, similarly there must be a corresponding, matching '**Blocked Receive**' command in the source code of the receiving or distinction processor

General format of 'Blocked Send' command

Blocked Send (Destination PE address, Variable-name or message, Size)

Address of the destination processor - the processor to which the message or data to be send

Variable (data) name or message - the starting memory address of the data or message on the source or sending processor

Size (message length) - the number of bytes to send

General format of 'Blocked Receive' command

Blocked Receive (Source PE address, Variable-name or message, Size)

Address of the source processor - tells the destination processor about the source of the message

Variable (data) name or message - indicates the memory address on the destination where the received data variable will be stored

Size (message length) - the number of bytes to be received

Asynchronous Message Passing

A simple '**Send**' command is used explicitly by the user for sending data

Programmer puts '**Send**' command in the source code of the sending processor

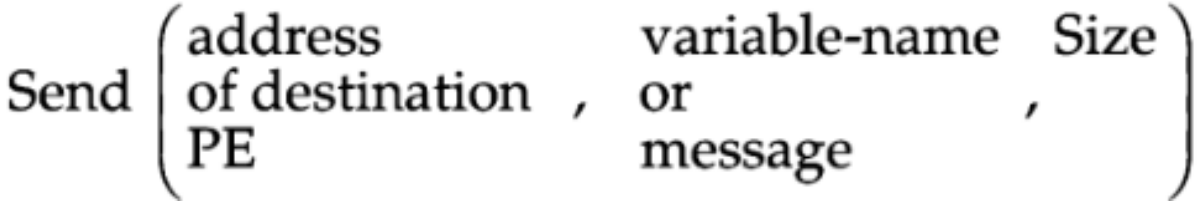
Sending processor sends data ⇒ continues its processing further ⇒ does not wait for ensuring about data reception

Corresponding, matching '**Receive**' command must be there, in the code of receiving process

'Send' and 'Receive' are used if ordering is not important and the system hardware reliably delivers messages

Non blocking ‘Send’ and ‘Receive’ ⇒ Asynchronous message passing system

General Format of 'Send' Command



Note: arguments are same as that of '**Blocked Send**' command

General Format of 'Receive' Command

$$\text{Receive}\left(\begin{array}{l}\text{address}\\ \text{of destination}\\ \text{PE}\end{array}\ ,\ \begin{array}{l}\text{variable-name}\\ \text{or}\\ \text{message}\end{array}\ ,\ \text{Size}\right)$$

Note: arguments are same as that of '**Blocked Receive**' command

Example

We need to add all elements of an array A[i] having n elements. Write a program for message passing multicomputer assuming only 2 PEs.

Example

Task 1 : Add A[1], …, A[N/2] to one PE

Task 2 : Add A[N/2 + 1] …, A[N] to second PE.

The procedure for the two CEs is as follows :

PE – 0

Read A[i] for i = 1, …, N
Send to PE1 A[i] for i = (N/2 + 1), …, N
Find Sum 0 = Sum of A[i], for i = 1, …, N/2
Receive from PE 1, Sum 1 computed by it
Sum = Sum 0 + Sum 1
Write Sum

Similarly, for PE – 1 :

Receive from PE 0, A[i] for i = (N/2 + 1), …, N
Find Sum 1 = Sum of A[i] for i = (N/2 + 1), …, N
Send Sum 1 to PE 0

Now, if we use the notation :

Send (1, A[N/2 + 1], N/2) then it will be equivalent to :

"Send to PE 1, N/2 **elements of the array, A starting from** $(N/2 + 1)^{th}$ **elements"**

Similarly the notation—**receive** (0, B[1], N/2) will be equivalent to saying :

"Receive from PE 0, N/2 values and store them starting at B[1]"

Example (contd…)

This example can also be extended to develop a parallel program on the parallel architecture with multiple processors also

The array of numbers is divided equally among multiple processors and each processor will add its own array set

Multiple programs are running on multiple processors

Here the programming mode can be modified by writing **only one program** instead of multiple programs

The same program is executed on multiple processors but each processor does computation only on its allotted data variable values ⇒ the programming approach is called as 'Single Program with Multiple Data' (SPMD)

Advantages of message passing programming model

These programs are portable, from one machine to another

These programs that are written using libraries such as MPI (Message Passing Interface) and PVM (Parallel Virtual Machine) are platform independent

Such programs can run on all parallel architectures such as multiprocessors, MPP (Massively Parallel Processors), SMP (Symmetric Multiprocessors) etc.

Shared Memory Programming

Shared Memory Programming

This model is suitable for the shared memory parallel computer in which multiple processors share common global memory

Programmer uses this model for writing parallel programs ⇒ consists of processes which share variable in the global memory

In this model, there is no need to explicitly allot data for different processes because common data are stored in shared main memory

Multiple cooperating processes which share variables i.e., processes have a single shared address space

Shared Memory Programming

In this model separate processes are created and task workload are assigned to each process

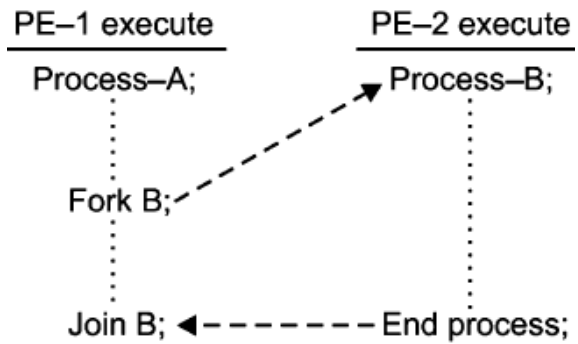
Main program uses '**fork**' to create processes and then assigns processes to multiple processors

Each PE independently carries out the work distributed to it

When all PEs execute their jobs then finally main program is executed which synchronizes all processes

Every process can combine to main program after finishing its work by the use of '**join**' function

Shared Memory Programming - Example



Example

Here, the process-A starts execution on processor 1 (PE-1), during its execution it comes to fork(), another child process ⇒ process B is created and it starts execution simultaneously on PE-2

Again when process-A encounters join B then it expects the result from process B, at this stage, if process B is yet not completed, then process-A has to wait otherwise it can continue

But we have to take care while accessing a shared memory variable

Each process must not be able to access shared variable at the same time ⇒ this is achieved by using a 'lock mechanism'

Example

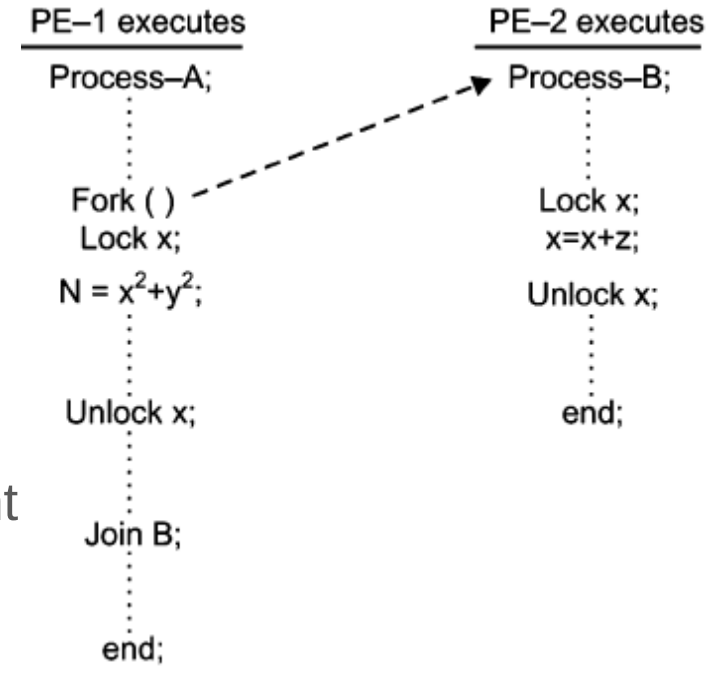
Processor 1 (PE-1) starts the execution of process-A and when it encounters fork(), then process-B is created on PE-2

Now, concurrent execution of both the processes takes place

Both the processes use 'lock x' statement

Process which first executes the lock statement gets the control of the shared variable x

Only after unlocking it, by that particular process, other process may get access to it



Data Parallel Programming

Data Parallel Programming

This model exploits data parallelism

Suitable for SIMD (Single Instruction Multiple Data) or SPMD (Single Program Multiple Data) parallel machines

Important feature of this model is that the same operation is performed parallely on different elements of the data structure such as arrays, matrices etc.

Multiple processors of the parallel machine execute the same program on different data elements of data structure

Parallel program has a single thread of control ⇒ In this model there is data parallelism but no control parallelism/functional parallelism

Control parallelism ⇒ simultaneous execution of different instruction streams

Data Parallel Programming

Suitable for applications in which partitioning and distribution of data is possible

The data parallel languages help the programmer to automatically perform repetitive operations on arrays, matrices like data structures

Data parallel languages: FORTRAN-90, High Performance Fortran (HPF) etc.

In serial languages the programmer has to specify everything, right from the operation to perform the partitioning of data structure, loops etc.

Programmer using data parallel language can only concentrate on the logic required to solve the given problem

Parallel Algorithms for Multiprocessors

Parallel Algorithms

Algorithm ⇒ a sequence of steps that take inputs from the user and after some computation, produces an output

In computer science/engineering, a **parallel algorithm**, as opposed to a traditional serial algorithm, is an algorithm which can do multiple operations in a given time

Parallel algorithm ⇒ an algorithm that can execute several instructions simultaneously on different processing devices and then combine all the individual outputs to produce the final result

Parallel Algorithms

It has been a tradition to describe serial algorithms in abstract machine models

Many computer science/engineering researchers have used a parallel random-access machine (PRAM) as a parallel abstract machine (shared-memory)

Many parallel algorithms are executed concurrently – though in general concurrent algorithms are a distinct concept – and thus these concepts are often conflated, with which aspect of an algorithm is parallel and which is concurrent not being clearly distinguished

Further, non-parallel, non-concurrent algorithms are often referred to as "sequential algorithms", by contrast with concurrent algorithms

Sequential Algorithm & Parallel Algorithm

Sequential Algorithm ⇒ an algorithm in which some consecutive steps of instructions are executed in a chronological order to solve a problem

Parallel Algorithm ⇒ the given problem is divided into sub-problems and are executed in parallel to get individual outputs

> Later on, these individual outputs are combined together to get the final desired output

May not be easy to divide a large problem into sub-problems ⇒ sub-problems may have data dependency among them

⇒ Processors need to communicate with each other to solve the problem

Classification of Parallel Algorithms

A parallel algorithm for the multiprocessor is a set of K concurrent processes which may operate simultaneously and co-operatively to solve given problem

When K=1, it becomes a sequential algorithm

In a task system, some points are there, where processes communicate with each other to ensure that a parallel algorithm works correctly and effectively ⇒ these points are called as interaction points ⇒ these points divide a process into stages

At the end of each stage a process may communicate with some other processes before the next stage of the computation is initiated

Synchronized Parallel Algorithms

It is a parallel algorithms consisting of processes with the property that, there exist a process such that some stage of the process is not activated until another process has completed a certain stage of its program

The synchronization can be performed using various synchronized primitives

e.g., To compute the matrix by maximum decomposition

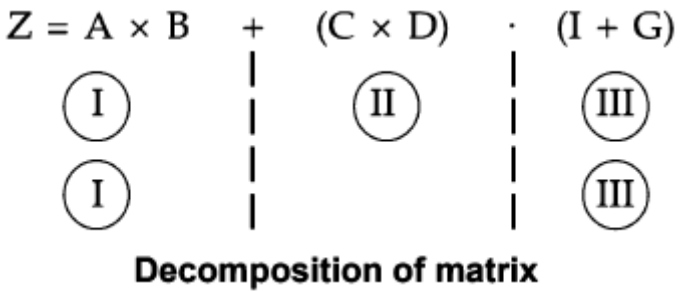
$$Z = A.B + (C*D).(I+G)$$

New parallel algorithms may be constructed

The matrix is decomposed into three different processes as P1, P2 and P3 processes

Synchronized Parallel Algorithms

It consists of different stages as



Here, Process-1 consists of two stages i.e., (A * B) and addition of (A * B) and (C * D) whereas Process-2 and Process-3 consists of one and two stage respectively, as (C * D) and (I + G) and multiplication of (C*D) and (I + G)

Synchronized Parallel Algorithms

Algorithm:

Here, the activation of second stage of process P3 is subject to the condition that process P2 is completed

Similarly, the second stage of P1 cannot be initiated unless the second stage of P3 is completed

Hence, the set of processes, P1, P2 and P3 is synchronized parallel algorithm

```
Var w, y : Shared real ;
Var Sw, Sy : Semaphore ;
initial Sw = Sy = 0
Cobegin
Process P1 : begin
        V ← A × B ;      // stage 1 of P1
        P(Sy) ;
        Z ← V + Y ;      // stage 2 of P2
        end
Process P2 : begin
        W ← C × D ;      // stage 1 of P2
        V (Sw) ;
        end
Process P3 : begin
        X ← I + G ;      // stage 1 of P3
        P (Sw) ;
        Y ← W + X ;      // stage 2 of P3
        V (Sy) ;
        end
Coend
```

Asynchronized Parallel Algorithms

In these algorithms, there is a set of global variables accessible to all processes

The communication between processes is achieved through global variables or shared data

When a stage of process is completed, the process reads some global variables

The process modifies a subset of the global variables and then activates the next stage or terminates itself based on the values of the variables read together with results obtained from the last stage

Asynchronized Parallel Algorithms

In asynchronous algorithm, there is no explicit dependency between the processes as in synchronous parallel algorithms which can be considered as one of the advantages of this algorithms

In this algorithm, processes never wait for inputs at any time but continue execution or terminate according to the information contained in global variables

Herein, the processes may be blocked from entering critical sections, which can be considered as one of it's disadvantages

Asynchronized Parallel Algorithms

Example

$$x_{i+1} = x_i - f'(x_i)\, f(x_i)$$

Here, an asynchronous iterative algorithm consisting of two processes, P1 and P2 and three global variables are defined

Note: process P1 updates variables V1 and V3, while process P2 updates V2

Algorithm :

```
function f, f' ;
Var V1, V2, V3 : shared real ;        // Global variables
Cobegin
Process P1 : begin
            While < termination criteria is not satisfied > do
            begin
                V1 <- (f(V3)) ;
                V3 <- V3 – V2·V2 ;
            end
        end P1
Process P2 : begin
            While < termination criteria is not satisfied > do
                V2 ← f'(V3) ;
            end P2
Coend
```

Asynchronized Parallel Algorithms

In this example, as soon as the process completes updating a global variable, it proceeds to the next updating by using the current values of relevant variables without any delay

Here, process P2 executes f'(V3) which is required by process P1

If the initial values of variable are:

$$V_1 = f(x_0)$$
$$V_2 = f'(x_0)$$
$$V_3 = x_1$$

Asynchronized Parallel Algorithms

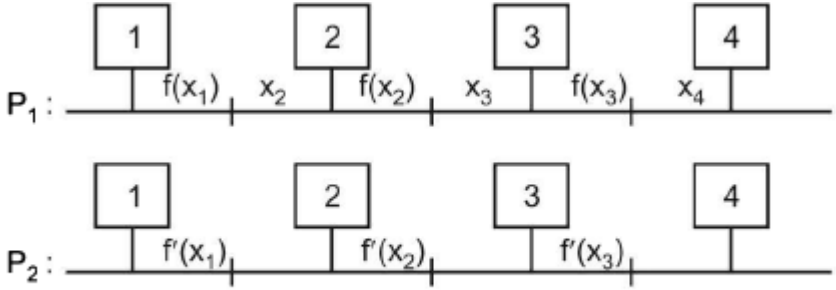
Then the sequence and time period of step completion for each iteration within each process may be illustrated as ⇒

Advantages of this algorithm is its general applicability

These algorithms are mainly used when decomposition of the processes is difficult

Disadvantages are: Critical sections are needed in the algorithms

The speed-up of the algorithm is quite limited



Timing diagram for Asynchronous Parallel Algorithm

Performance of Parallel Algorithms

Performance of Parallel Algorithms

The effectiveness of a multiprocessor for a synchronized iterative algorithm depends on the performance feature of the algorithm

Memory interference affects the performance in tightly coupled systems

In case of loosely coupled systems, the cost of the interprocess communication is the dominant factor

Synchronization and interprocess communication affects the performance of a synchronized iterative algorithm

Problems Based on Parallel Algorithms

Problems Based on Parallel Algorithms

Searching

Prime Numbers

Graph Algorithms

Quick Sort

Searching

Sequential Search

In sequential computing, the scanning of sequence S and comparing x with every element in S is done until either a number equal to x is found or the sequence is exhausted without success

takes O(n) time

Searching

Parallel Search

Searching on a tree, we use a binary tree network with n leaf nodes (processors)

Total number of nodes ==>number of processors = 2n-1

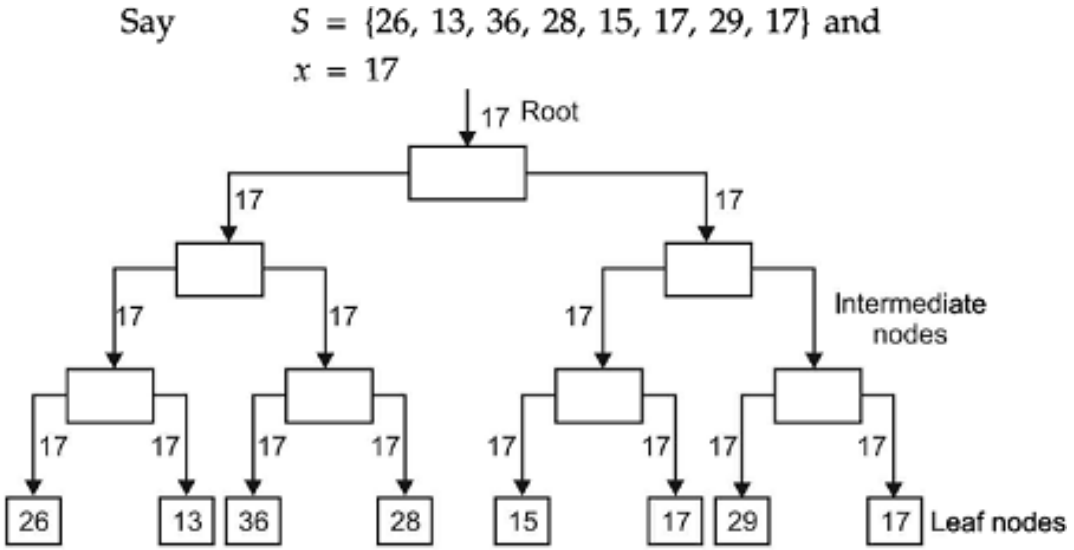
The sequence S = {S1, S2, ..., Sn} which has to be searched is stored in the leaf nodes

The processor Li stores the element Si

Searching

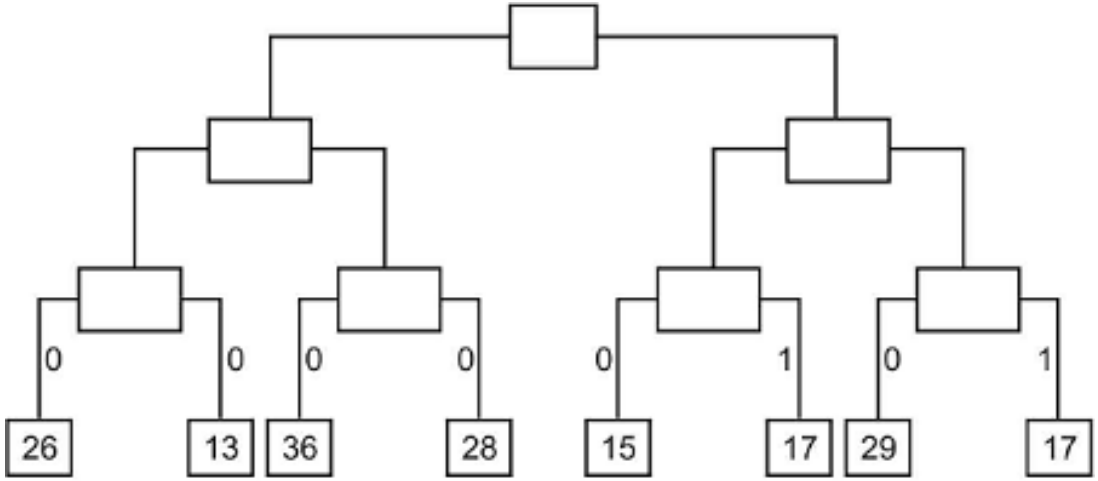
The algorithm has 3-stages

- The root-node reads x and passes it to its two child nodes

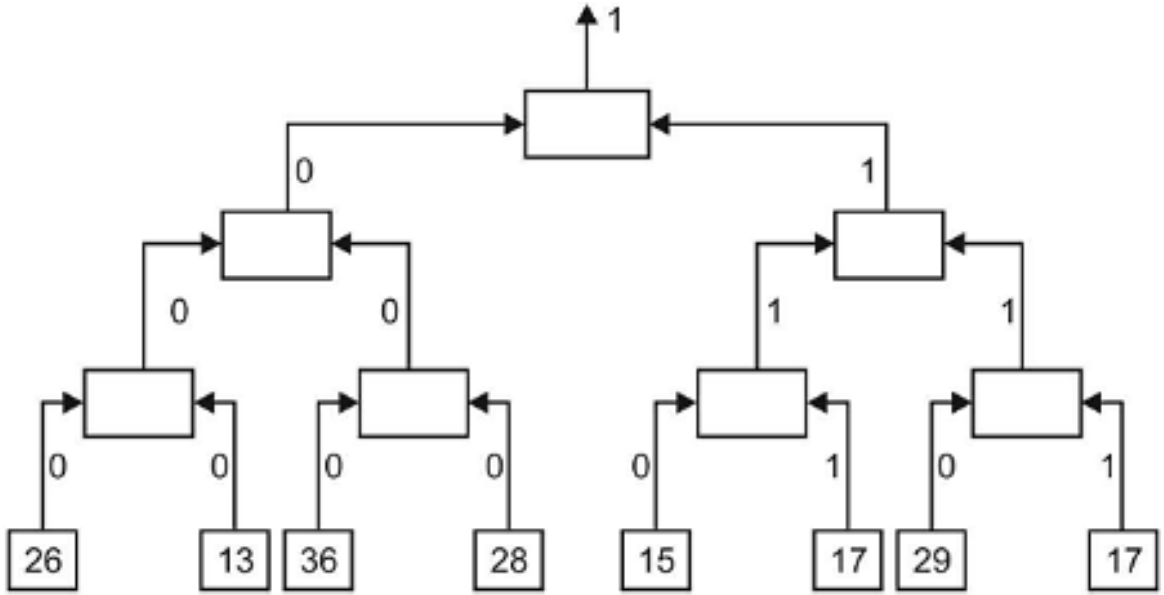


Searching

- Every leaf node Li compares the element x with Si simultaneously



Searching



- The result (1 or 0) of search operation at the leaf nodes are combined

Searching

The algorithm takes O(log n) time to go down the tree, constant time to go back up the tree

Time taken by algorithm to search for an element is O(log n)

Sequential vs. Parallel

O(n) O(logn n)

Problems Based on Parallel Algorithms

Prime Numbers

Graph Algorithms

Quick Sort

Left as exercise

Parallel Random Access Machine (PRAM) Model

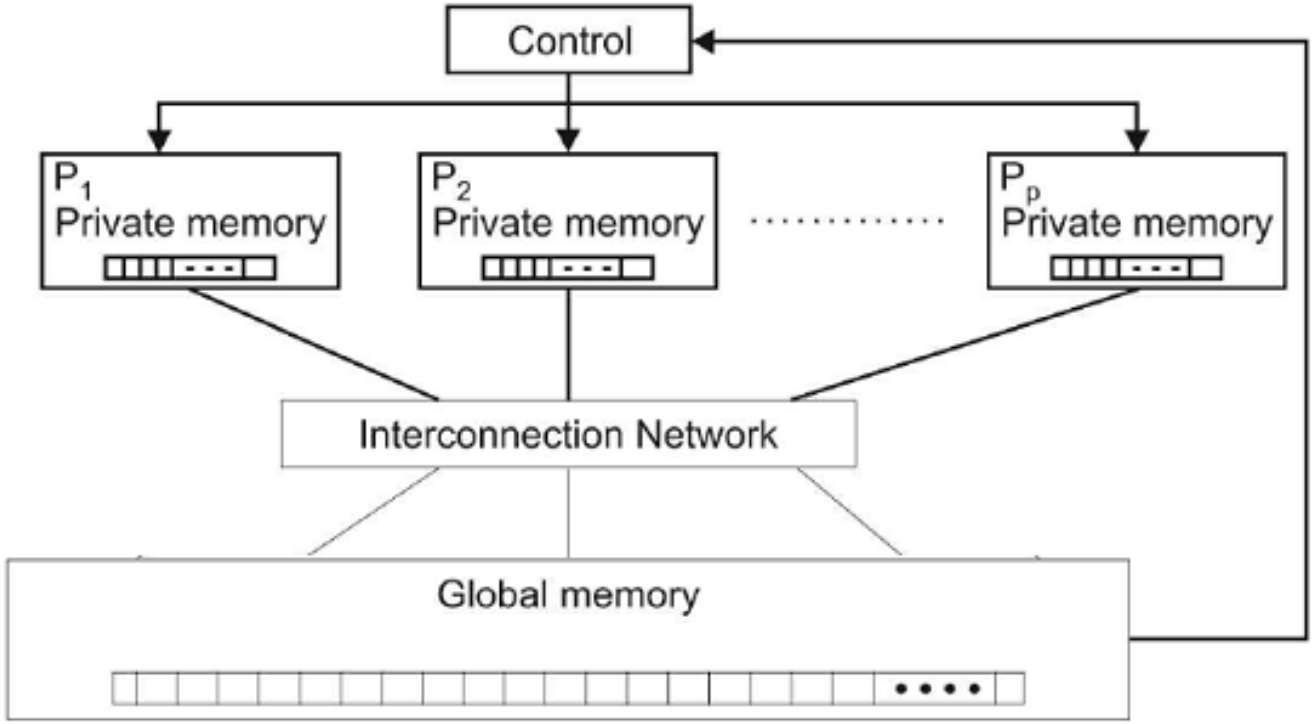
Parallel Random Access Machine (PRAM) Model

Allows parallel algorithm designers to treat processing power as an unlimited resource ⇒ programmers of computers with virtual memory are allowed to treat memory as an ultimate resource

PRAM model is simple and it ignores the complexity of interprocessor communication

As communication complexity is not an issue, the designer of PRAM algorithm can focus on parallelism inherent in particular computation

PRAM Model



PRAM Model

PRAM Model

There are 4 subclasses of the PRAM model, provided to define how simultaneously reading and writing should be handled

Exclusive Read, Exclusive Write (EREW PRAM) - in this model, a memory location can only be accessed by one processor at a time, whether for reading the contents or writing new values ⇒ most restrictive model

Concurrent Read, Exclusive Write (CREW PRAM) - in this model, a memory location can be accessed by more than one processor simultaneously but only for reading the contents of the location ⇒ all accesses to write new values can only be done one at a time

PRAM Model

Exclusive Read, Concurrent Write (ERCW PRAM) - in this model, a memory location can be accessed simultaneously for writing but not for simultaneous reading ⇒ this model usually not considered

Concurrent Read, Concurrent Write (CRCW PRAM) - in this model, a memory location can be accessed simultaneously by different processors, for either reading or writing ⇒ this is the most powerful model

Write conflict in PRAM Models

In the models with simultaneous writing (ERCW & CRCW), additional specification is necessary to resolve the conflicts

Common - simultaneous writing is only allowed if all the new values to be written are same ⇒ then the result is defined as this value

Arbitrary - one processor is selected to perform the write operation and all other processors are inhibited

Priority - similar to arbitrary but here a predefined priority order is established to select the processor that will succeed, the rest of processors failing ⇒ the predefined order is by process identification number

Sum - final result is the arithmetic sum of values being written by the individual processors

PRAM Algorithms

As PRAM algorithm begins with only a single processor active ==>two phases

First phase - sufficient number of processors are activated

Second phase - these activated processors perform computation in parallel

Parallel Programming Languages

Parallel Programming Languages

Parallel programming is the design, implementation and tuning of parallel computer programs which take the advantage of parallel computing system

It also refers to the application of the parallel programming models to existing serial programs

E.g., FORTRAN-90, OCCAN, C-Linda and CCC

FORTRAN-90

A new programming language

Permits efficient execution of code on conventional as well as on vector processors

But it is less efficient on parallel computers

FORTRAN-90 New Features

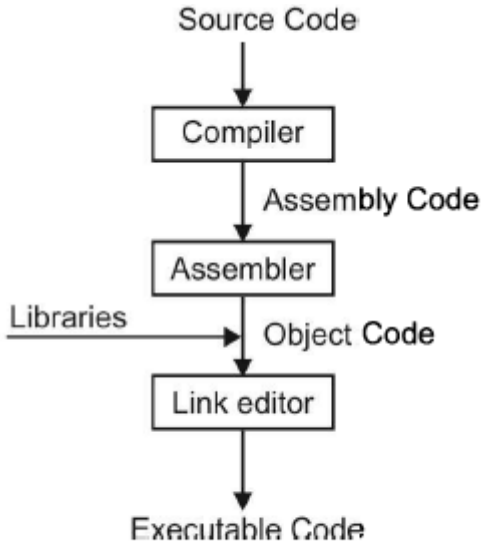
Free format on source code:- In Fortran-90, we can use either the Fortran 77 input format or free format

Dynamic allocation and pointers:- Dynamic memory allocation by means of the ALLOCATOABLE attribute and the ALLOCATE and DEALLOCATE statements

Pointer attribute, pointer assignment, and NULLIFY statements to facilitate the creation and manipulation of dynamic data structures

User defined data types:- We can define own composite data types similar to struct in C

The Fortran 90 program can be compiled using the following steps:



Fortran–90 program compilation steps.

The features of following programming languages are left as exercise

- OCCAN
- C-Linda and
- CCC